provide utilities to run a mock web server. Open the build.gradle file of the Order Service project and add the missing dependency.

Listing 8.27 Adding test dependency for OkHttp `MockWebServer`

```
dependencies {
  ...
  testImplementation 'com.squareup.okhttp3:mockwebserver'
}
```

Let's start by testing the `BookClient` class.

8.5.1 Testing REST clients with a mock web server

The OkHttp project provides a mock web server that you can use to test HTTP-based request/response interactions with a service downstream. `BookClient` returns a `Mono<Book>` object, so you can use the convenient utilities provided by Project Reactor for testing reactive applications. The `StepVerifier` object lets you process reactive streams and write assertions in steps through a fluent API.

First, let's set up the mock web server and configure `WebClient` to use it in a new `BookClientTests` class.

Listing 8.28 Preparing the test setup with a mock web server

```
package com.polarbookshop.orderservice.book;

import java.io.IOException;
import okhttp3.mockwebserver.MockWebServer;
import org.junit.jupiter.api.*;
import org.springframework.web.reactive.function.client.WebClient;

class BookClientTests {
  private MockWebServer mockWebServer;
  private BookClient bookClient;

  @BeforeEach
  void setup() throws IOException {
```